

URL Shortener Microservices

Thiết kế, triển khai và vận hành hệ thống rút gọn URL theo kiến trúc Microservices

Phát triển Phần mềm theo Kiến trúc Microservices — SE361.Q21

University of Information Technology — VNU-HCM

2026

Thông tin nhóm thực hiện

Thành viên	nhóm
Nguyễn Mỹ Thông	23521527
Đặng Trần Anh Hào	23520444
Trần Nam Phong	23521172

Môn học & lớp

Phát triển Phần mềm theo Kiến trúc Microservices

SE361.Q21

Giảng viên hướng dẫn

Đặng Việt Dũng

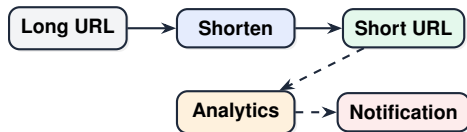
Giới thiệu đề tài

Đề tài

Xây dựng hệ thống rút gọn URL theo kiến trúc microservices, hỗ trợ xác thực người dùng, tạo short URL, redirect, thống kê click và thông báo sự kiện.

Động lực

Redirect là luồng đọc tần suất cao; analytics và notification là tác vụ phụ trợ cần tách khỏi request path chính.



Trọng tâm báo cáo là cách hệ thống hiện thực các nguyên lý microservices: bounded context, database-per-service, event-driven, resilience, observability và triển khai container/Kubernetes.

Đề cương nhắm tới

DDD & Bounded Context

REST + Async Messaging

Database per Service

Security + Resilience

Docker + Monitoring + Testing

Dự án đã hiện thực

5 binary Go độc lập: gateway, user, url, analytics, notification.

HTTP/JSON qua gateway; RabbitMQ topic exchange cho click, created, deleted, milestone.

4 PostgreSQL database riêng; Redis chỉ làm cache và rate-limit store.

Prometheus/Grafana, circuit breaker, k6 script, smoke/e2e scripts.

Bài toán: Độ phức tạp trong hệ thống rút gọn URL

Hạn chế của thiết kế monolith đơn giản

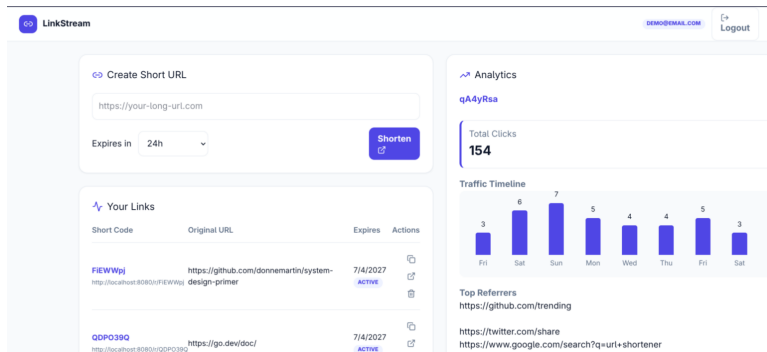
- ▶ Redirect, analytics và notification cùng nằm trên một request path.
- ▶ Analytics lỗi có thể kéo chậm redirect.
- ▶ Việc mở rộng phải áp dụng cho toàn bộ hệ thống dù tải chủ yếu đến từ redirect.
- ▶ DB chung dễ sinh join chéo và coupling schema.

Cách tiếp cận microservices

- ▶ Redirect là luồng xử lý tần suất cao, cần ưu tiên cache và độ trễ thấp.
- ▶ Analytics/notification xử lý bất đồng bộ qua event.
- ▶ Mỗi service sở hữu dữ liệu của mình.
- ▶ Gateway bảo vệ hệ thống bằng auth, rate limit, circuit breaker.

Mục tiêu thiết kế: redirect nhanh khi phụ trợ lỗi một phần, nhưng vẫn thu thập click và gửi thông báo bằng event.

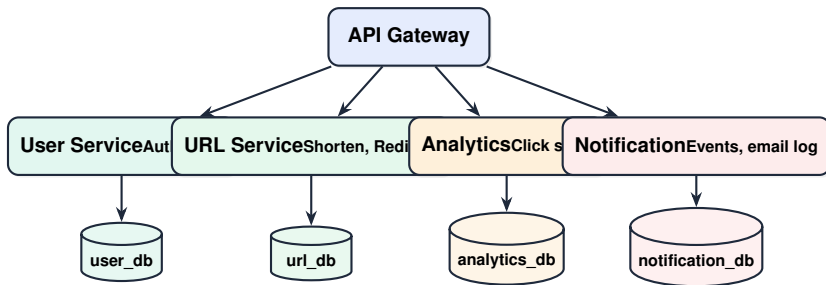
Giao diện người dùng: luồng rút gọn và theo dõi URL



Luồng sản phẩm

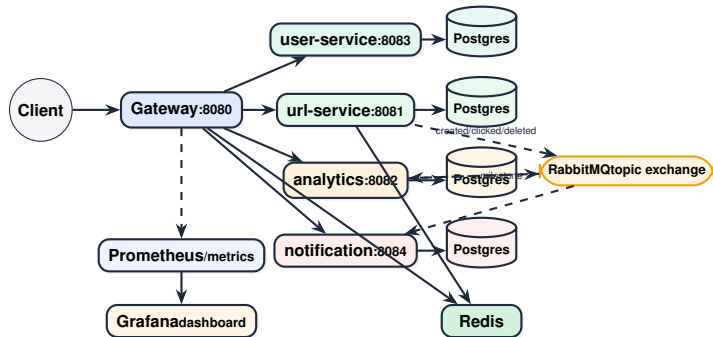
- ▶ Đăng ký / đăng nhập.
- ▶ Rút gọn URL.
- ▶ Danh sách URL theo user.
- ▶ Redirect qua short code.
- ▶ Thống kê click và notification.

Tách Bounded Context

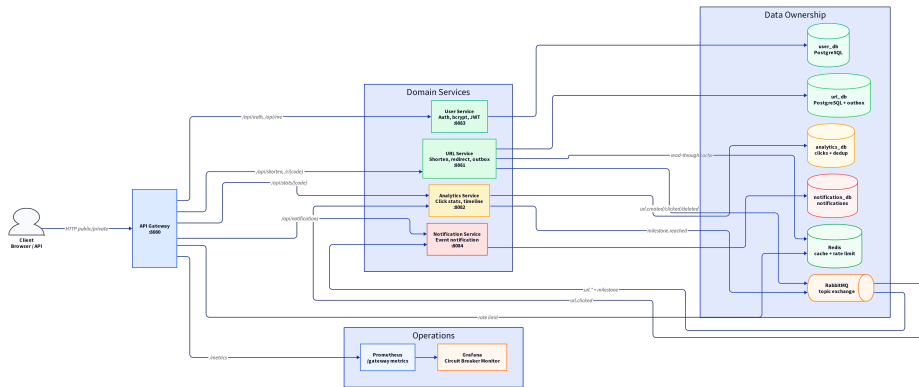


Ranh giới kiến trúc: monorepo chỉ là cách tổ chức source code; ranh giới runtime là binary, container, API và database riêng.

Kiến trúc tổng quan



D2 Architecture Map: tổng quan hệ thống

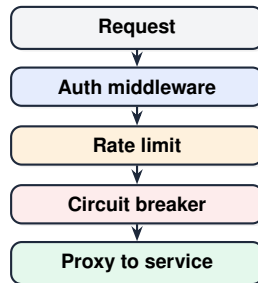


Sơ đồ D2 thể hiện runtime boundary, data ownership và operations của hệ thống microservices.

API Gateway: cửa vào duy nhất

Chức năng tại lớp biên hệ thống

- ▶ Reverse proxy route theo path.
- ▶ JWT validation cho route private.
- ▶ Rate limit theo IP bằng Redis.
- ▶ Circuit breaker từng upstream.
- ▶ Gắn X-Correlation-ID.
- ▶ Expose `/metrics` cho Prometheus.



REST API surface

Nhóm	Gateway endpoint	Trách nhiệm service
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/me	Register, login, bcrypt, JWT, profile
URL	POST /api/shorten, GET /api/urls, DELETE /api/urls/{code}	Short code, ownership, deactivate, out-box
Redirect	GET /r/{code}	Redis cache, DB fallback, 301 redirect, click event
Analytics	GET /api/stats/{code}, timeline	Total click, 24h, 7d, referer, time bucket
Notification	GET /api/notifications	List notification theo user, cursor paging

Thiết kế RESTful thực tiễn: route public/private rõ, list API có cursor pagination, limit được clamp, body auth giới hạn 1MB.

URL Service: tối ưu luồng redirect tần suất cao

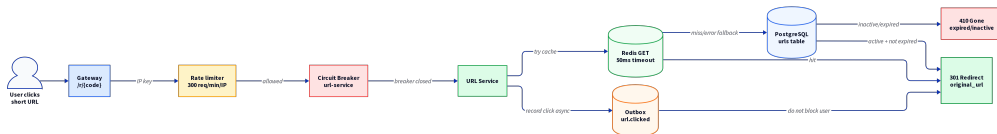
Shorten

- ▶ Validate URL chỉ nhận `http/https`.
- ▶ Sinh base62 7 ký tự bằng `crypto/rand`.
- ▶ Không gian mã: $62^7 \approx 3.5$ nghìn tỷ.
- ▶ Unique constraint + retry collision.
- ▶ Ghi `url.created` vào outbox trong transaction.

Redirect

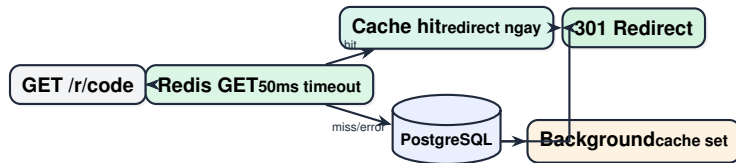
- ▶ Redis read-through cache, timeout 50ms.
- ▶ Redis lỗi/cache miss thì fallback PostgreSQL.
- ▶ Check inactive/expired trả 410.
- ▶ Hash IP, ghi user-agent/referer.
- ▶ Click analytics đi qua event, không block bởi consumer.

D2 Redirect Hot Path: luồng chịu tải cao nhất



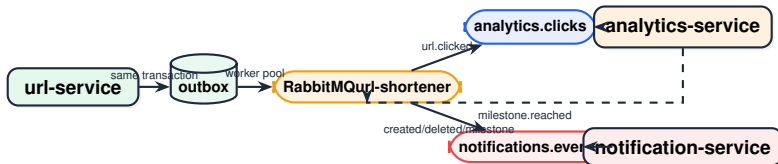
Redirect không chờ analytics consumer. Click event được ghi bất đồng bộ; người dùng nhận redirect nhanh nhất có thể.

Redis: nhanh nhưng không phải source of truth



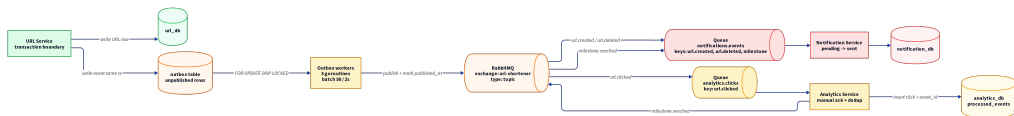
Ngữ nghĩa khi Redis lỗi: redirect fallback về PostgreSQL. Hệ thống chậm hơn nhưng không mất dữ liệu vì PostgreSQL là nguồn dữ liệu chính.

Event-driven: tách analytics và notification khỏi redirect



Event payload chứa đủ dữ liệu để consumer xử lý, giảm gọi ngược service khác. Đây là trade-off denormalization có chủ đích để giảm coupling.

D2 Event Pipeline: Outbox + RabbitMQ + Consumers



Outbox Pattern: đảm bảo lưu trữ event khi broker lỗi

Nếu publish trực tiếp

- ▶ Ghi URL thành công.
- ▶ RabbitMQ timeout đúng lúc publish.
- ▶ DB có dữ liệu nhưng analytics không biết.
- ▶ Có nguy cơ mất event và khó đồng bộ lại giữa các service.

Cách triển khai trong hệ thống

- ▶ Ghi URL và event vào `outbox` cùng transaction.
- ▶ Worker poll mỗi 2 giây, batch 50.
- ▶ 3 worker goroutines.
- ▶ `FOR UPDATE SKIP LOCKED` tránh đụng nhau.
- ▶ Publish xong mới set `published_at`.

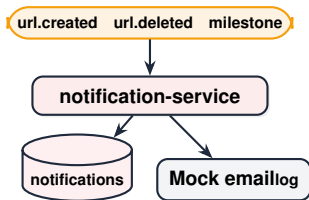
Consumer behavior

- ▶ Consume `url.clicked` từ RabbitMQ.
- ▶ Manual ack/nack.
- ▶ QOS prefetch 1 để kiểm soát áp lực.
- ▶ Invalid JSON hoặc thiếu event id thì ack và discard.
- ▶ Deduplicate bằng bảng `processed_events`.

Stats API

- ▶ Total clicks.
- ▶ Clicks last 24h và last 7d.
- ▶ Top referers.
- ▶ Timeline theo day hoặc hour.
- ▶ Query song song bằng errgroup.
- ▶ Milestone: 10, 100, 1000 clicks.

Notification Service: cô lập tác vụ phụ trợ



Chi tiết vận hành

- ▶ Consumer không làm URL Service chậm.
- ▶ Notification lưu trạng thái `pending`, `sent`, `sent_at`.
- ▶ List notification theo user có cursor pagination.
- ▶ Event `url.created` chứa email để tránh gọi ngược `user-service`.
- ▶ Mock email cho phép kiểm thử luồng gửi thông báo mà không cần tích hợp SMTP thực tế.

Security: các lớp bảo vệ chính

Identity

Register/login, email
unique, password tối thiểu
8 ký tự, bcrypt hash.

JWT

HS256, claims
sub/email/iss/iat/exp,
verify signing method
HMAC.

Data protection

Không lưu raw password,
hash IP trước khi đưa vào
analytics, ownership check
khi delete URL.

Phân tích kỹ thuật 1: giảm rủi ro timing attack khi login

Vấn đề bảo mật tinh vi

- ▶ Nếu email không tồn tại và trả lỗi ngay, response chỉ mất vài ms.
- ▶ Nếu email tồn tại nhưng password sai, bcrypt cost 12 mất khoảng hàng trăm ms.
- ▶ Kẻ tấn công có thể đo độ trễ phản hồi để suy luận email tồn tại trong cơ sở dữ liệu.

Cách hệ thống xử lý

- ▶ Login luôn trả cùng thông điệp: `invalid credentials`.
- ▶ Khi không tìm thấy user, service vẫn chạy bcrypt verify trên dummy hash hợp lệ cost 12.
- ▶ Mục tiêu là giảm chênh lệch timing giữa “email không tồn tại” và “password sai”.
- ▶ Implement tại `services/user-service/handler.go`.

Phân tích kỹ thuật 2: an toàn đồng thời trong Outbox

Tăng throughput bằng worker pool

- ▶ Outbox coordinator chạy 3 worker goroutines.
- ▶ Mỗi lần poll lấy tối đa 50 event.
- ▶ Poll interval 2s.
- ▶ DB claim event bằng FOR UPDATE SKIP LOCKED và locked_until.

An toàn với RabbitMQ channel

- ▶ Nhiều goroutine cùng publish qua một AMQP channel có rủi ro race.
- ▶ `amqp.Publisher` bọc `PublishWithContext` bằng `sync.Mutex`.
- ▶ Trade-off: publish được serialize để đảm bảo correctness, nhưng vẫn giữ worker pool cho phần fetch/mark/pipeline xử lý event.
- ▶ Implement tại `url-service/publisher.go`.

Phân tích kỹ thuật 3: cursor pagination với LIMIT + 1

Vì sao không dùng OFFSET

- ▶ OFFSET càng sâu càng chậm vì DB phải bỏ qua nhiều dòng.
- ▶ Với user có nhiều URL, list page sau sẽ đắt dần.
- ▶ COUNT(*) để biết còn trang sau cũng tốn thêm query.

Cách hệ thống làm

- ▶ API nhận cursor after thay vì offset.
- ▶ Query dùng `id < afterID, ORDER BY id DESC`.
- ▶ Fetch `limit + 1`; nếu dư 1 dòng thì biết có trang tiếp theo.
- ▶ Cắt dòng thừa và trả `next_cursor`, không cần COUNT(*) .
- ▶ Implement tại `url-service/store.go` và `service.go`.

Phân tích kỹ thuật 4: các primitive tự triển khai tại Gateway

Circuit Breaker tự triển khai

- ▶ State machine: CLOSED, OPEN, HALF_OPEN.
- ▶ Thread-safe bằng `sync.Mutex`.
- ▶ Chỉ cho một probe ở HALF_OPEN qua `halfOpenProbe`.
- ▶ Callback state-change cập nhật Prometheus metrics.

Rate Limiter Redis fail-open

- ▶ Fixed-window counter bằng INCR + EXPIRE.
- ▶ Mỗi check có timeout 100ms.
- ▶ Nếu Redis lỗi, gateway **fail-open**: cho request đi qua thay vì làm sập API vì cache/rate-limit store.
- ▶ Implement tại `gateway/ratelimit.go`.

Phân tích kỹ thuật 5: graceful shutdown cho cloud-native runtime

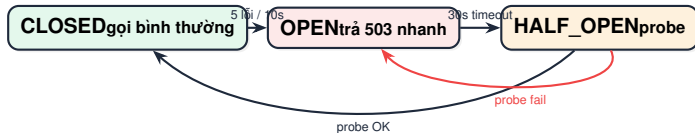
Khi Kubernetes rolling update

- ▶ Pod nhận SIGTERM trước khi bị kill.
- ▶ Service cần ngừng nhận request mới và cho request hiện tại kết thúc.
- ▶ URL service còn phải dừng outbox polling qua context cancellation.

Cách triển khai hiện tại

- ▶ Các service listen SIGINT/SIGTERM.
- ▶ HTTP server dùng `Shutdown(...)` để drain connection.
- ▶ URL service cancel root context cho outbox coordinator và dùng shutdown timeout 10s.
- ▶ DB, Redis, RabbitMQ connection được `Close()` bằng `defer` sau khi main thoát.

Resilience: cô lập lỗi downstream tại gateway



Circuit breaker: giảm cascading failure, tránh request treo hàng loạt khi upstream mất khả dụng.

Rate limit: shorten 10 request/phút/IP, redirect 300 request/phút/IP; Redis lỗi thì fail-open.

Scalability: mở rộng theo nút nghẽn hệ thống

Redirect đọc nhiều

- ▶ Cache Redis giảm query DB.
- ▶ Gateway và url-service stateless ở tầng HTTP.
- ▶ Có thể tăng replica cho gateway/url-service.
- ▶ DB index theo `short_code`.

Async workload

- ▶ Analytics/notification tách khỏi request path.
- ▶ Outbox worker pool 3 goroutines.
- ▶ RabbitMQ durable queue hấp thụ spike.
- ▶ Milestone có unique constraint chống phát nhiều lần.

Horizontal scaling: Docker Compose có overlay scale cho `url-service=3`; Kubernetes deployment triển khai `url-service` 3 replicas và `gateway` 2 replicas qua Service discovery.

Database per Service

Service	Database	Bảng/ý nghĩa chính
user-service	user_db	users: email unique, password hash, created at
url-service	url_db	urls, outbox; index short_code, user_id, created_at
analytics	analytics_db	clicks, milestones, processed_events dedup
notification	notification_db	notifications: payload JSONB, status, sent at

Nguyên tắc nhất quán dữ liệu: không cross-service join, không shared schema. Consistency giữa services đạt bằng domain event và eventual consistency.

Database Boundary: tách vật lý, không chỉ tách logical

Docker Compose

- ▶ 4 PostgreSQL containers độc lập.
- ▶ 4 database name và credential riêng.
- ▶ 4 volume riêng: url_db_data, analytics_db_data, user_db_data, notification_db_data.
- ▶ Mỗi app chỉ nhận DATABASE_URL của service đó.

Kubernetes

- ▶ 4 DB Deployments/Services: url-db, analytics-db, user-db, notification-db.
- ▶ ConfigMap có 4 DSN riêng: URL_DATABASE_URL, ANALYTICS_DATABASE_URL, USER_DATABASE_URL, NOTIFICATION_DATABASE_URL.
- ▶ App Deployment chỉ mount DSN tương ứng vào DATABASE_URL.

CRUD Ownership: mỗi service chỉ thao tác bảng của mình

Service	CRUD boundary	Không phụ thuộc DB khác
User	INSERT users, SELECT users WHERE email; bcrypt/JWT nằm trong user-service	Không đọc urls, clicks, notifications
URL	INSERT urls, SELECT urls WHERE short_code, list theo user_id, soft delete bằng UPDATE is_active=false	Ownership check nằm trong điều kiện SQL short_code + user_id
Analytics	INSERT clicks, query stats/timeline, processed_events, milestones	Không join sang urls; nhận short_code qua event
Notification	INSERT notifications, update status pending -> sent, list theo user_id	Không join sang users; nhận payload qua event

Schema Ownership: không có foreign key xuyên service

Denormalized identifiers

- ▶ `url_db.urls` lưu `user_id`, `user_email` để URL service không join user DB.
- ▶ `analytics_db.clicks` lưu `short_code` để analytics không join URL DB.
- ▶ `notification_db.notifications` lưu `user_id` và payload JSONB để không join user/url DB.

Consistency qua event

- ▶ URL mutation ghi event vào outbox cùng transaction.
- ▶ Analytics nhận `url.clicked` và tự lưu click local.
- ▶ Notification nhận `url.created`, `url.deleted`, `milestone.reached`.
- ▶ Hệ thống chấp nhận eventual consistency thay vì shared transaction xuyên DB.

Database Indexing: thiết kế theo query path

Query path	Index/constraint	Mục đích
Redirect by code	idx_urls_short_code, UNIQUE(short_code)	Lookup nhanh và chống collision mã ngắn
List URLs by user	idx_urls_user_id_created	Hỗ trợ cursor pagination theo owner
Outbox publish	idx_outbox_unpublished, idx_outbox_unpublished_unlocked	Worker claim event chưa publish hiệu quả
Analytics stats	idx_clicks_short_code_time, idx_clicks_referer	Count/timeline/top referer theo short code
Milestone idempotency	UNIQUE(short_code, milestone)	Tránh phát sinh nhiều milestone trùng
Notification list	idx_notifications_user_created	List notification theo user và thời gian

Testing: kiểm chứng trên nhiều mức độ

Unit tests

gateway, events, user,
URL, analytics,
notification.

Smoke/E2E

health ports 8080–8084,
register, login, shorten,
redirect, stats, notification,
delete.

Load test

k6 redirect workload,
monitor circuit breaker
bằng Grafana.

Các case quan trọng: JWT middleware, path rewrite, rate limit, circuit breaker state, invalid event ack/discard, analytics dedup, milestone unique, deleted URL trả 410.

Giới hạn của mô hình delivery

- ▶ RabbitMQ + outbox thực tế là at-least-once.
- ▶ Message có thể bị redeliver.
- ▶ Network partition không cho exactly-once miễn phí.

Cách giảm tác hại

- ▶ Analytics dùng `processed_events` để dedup.
- ▶ Milestone có unique constraint.
- ▶ Outbox tránh mất event khi broker lỗi.
- ▶ Notification duplicate là hạn chế đã nhận diện.

Delivery model: hệ thống chọn at-least-once delivery kết hợp idempotent consumer ở nơi quan trọng, thay vì tuyên bố exactly-once không thực tế trong môi trường distributed system.

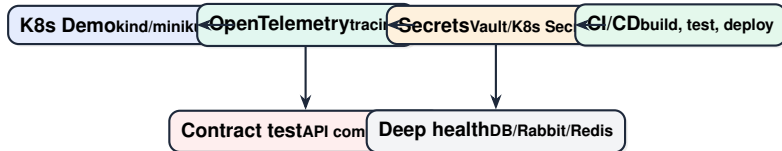
Phạm vi triển khai hiện tại

- ▶ Local Docker Compose chạy toàn stack.
- ▶ Kubernetes manifests cho kind/minikube.
- ▶ Microservice boundary rõ.
- ▶ Có cache, broker, DB riêng, metrics.
- ▶ Có kịch bản mô phỏng lỗi bằng circuit breaker.

Chưa production-ready hoàn toàn

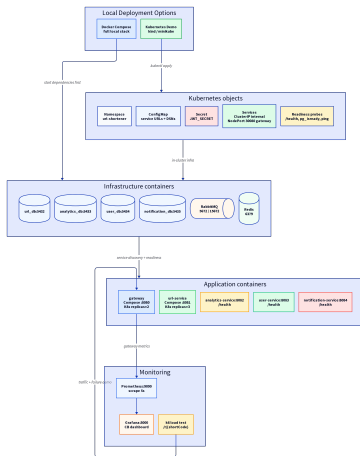
- ▶ K8s hiện hướng tới local cluster, chưa production-grade managed cluster.
- ▶ Chưa OpenTelemetry tracing/Jaeger.
- ▶ Metrics chủ yếu ở gateway.
- ▶ Secret hiện là giá trị mẫu, chưa tích hợp Vault/Secret Manager.
- ▶ Chưa CI/CD workflow và contract test đầy đủ.

Triển khai Kubernetes & lộ trình production



Repo đã có manifest Kubernetes cho môi trường local: Namespace, ConfigMap, Secret, Deployments, Services, readiness probes, url-service 3 replicas, gateway 2 replicas và NodePort 30080. Lộ trình production còn lại là HPA, Ingress/TLS, managed secrets, tracing và CI/CD.

D2 Deployment & Observability: chạy được cả stack



Ứng dụng stateless được scale

- ▶ Namespace riêng: url-shortener.
- ▶ url-service: replicas: 3.
- ▶ gateway: replicas: 2.
- ▶ Các service nội bộ dùng ClusterIP.
- ▶ Gateway expose qua NodePort 30080.

Config và readiness

- ▶ ConfigMap: service URLs, Redis/RabbitMQ URLs, DB DSNs.
- ▶ Secret: JWT_SECRET.
- ▶ App readiness probe gọi /health.
- ▶ Infra readiness: pg_isready, redis-cli ping, rabbitmq-diagnostics ping.
- ▶ E2E chạy qua gateway NodePort 30080.

Infrastructure first

- ▶ 4 PostgreSQL containers có `pg_isready`.
- ▶ RabbitMQ dùng `rabbitmq-diagnostics ping`.
- ▶ Redis dùng `redis-cli ping`.
- ▶ Volumes riêng cho DB, RabbitMQ, Redis, Prometheus, Grafana.

App depends on healthy deps

- ▶ Service chỉ start khi DB/broker/cache healthy.
- ▶ Mỗi app expose `/health`.
- ▶ Gateway phụ thuộc 4 domain service healthy.
- ▶ Smoke test kiểm tra ports 8080–8084.

Prometheus metrics: giám sát ngoài logging

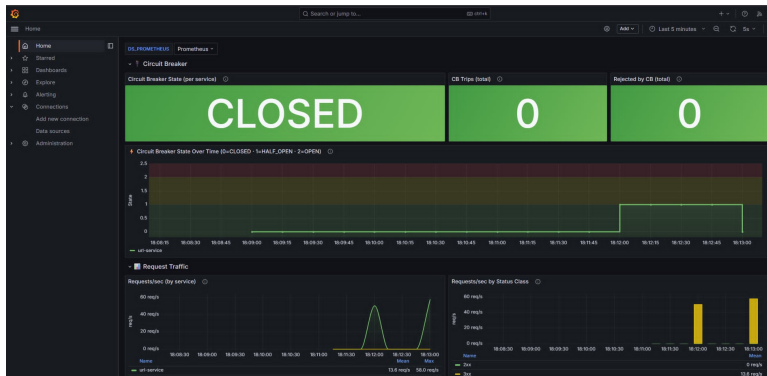
Gateway metrics

- ▶ gateway_requests_total
- ▶ gateway_request_duration_seconds
- ▶ gateway_circuit_breaker_state
- ▶ gateway_circuit_breaker_trips_total
- ▶ gateway_circuit_breaker_rejected_total

Grafana panels

- ▶ Circuit Breaker State: 0/1/2.
- ▶ Requests/sec by status class.
- ▶ Error Rate %.
- ▶ CB Trips total.

Grafana Dashboard: quan sát circuit breaker khi kiểm thử tải



Panels cần thể hiện

- ▶ Circuit breaker state.
- ▶ RPS theo status class.
- ▶ Error rate.
- ▶ CB trips / rejected requests.

Load Test Script: quy trình kiểm thử tải

Happy path

- 1 `docker compose up -d -build`
- 2 `./scripts/smoke_test.sh`
- 3 `k6 run scripts/load_test.js`
- 4 `k6 register, login, shorten URL.`
- 5 Traffic tập trung vào `/r/{shortCode}`.

Failure path

- 1 Đang load test thì stop url-service.
- 2 Gateway bắt đầu connection error.
- 3 Sau 5 lỗi trong 10s: breaker OPEN.
- 4 Request mới trả 503 nhanh, không chờ timeout downstream.
- 5 Start lại service, 30s sau HALF_OPEN probe.

Security Threat Model ngắn

Rủi ro	Cơ chế hiện tại	Hạn chế / mở rộng
Password leak	bcrypt hash, không lưu plaintext	Cần secret rotation, password policy mạnh hơn
JWT tampering	Verify HMAC signing method, issuer, expiry	Chưa có refresh/revocation
URL abuse/spam	Rate limit theo IP, body limit auth	Có thể thêm per-user quota và CAPTCHA
Privacy analytics	Hash IP trước khi lưu event	Salt config cần audit kỹ
Unauthorized delete	Ownership check bằng user id	Có thể thêm RBAC/team ownership
Secret exposure	Env config hiện dùng giá trị mẫu	Production cần Vault/K8s Secret

Các vấn đề thiết kế cần làm rõ

Vấn đề thiết kế	Quyết định kiến trúc
Broker tạm mất khả dụng	URL và event vẫn nằm trong DB/outbox; worker publish lại khi broker phục hồi.
Consumer nhận message trùng	Analytics dedup bằng <code>processed_events</code> ; milestone có unique constraint. Notification duplicate là giới hạn đã nhận diện.
Không dùng Saga	Use case không cần distributed transaction nhiều bước có compensation; Outbox + eventual consistency phù hợp hơn.
Cache stale sau delete	Delete URL set inactive trong DB và xóa cache Redis; DB vẫn là nguồn dữ liệu chính.
Auth ở gateway và ownership ở service	Gateway xác thực token tại edge; downstream vẫn nhận identity để enforce ownership trong domain service.
Horizontal scaling	Compose overlay hỗ trợ <code>url-service=3</code> ; Kubernetes triển khai <code>url-service</code> 3 replicas và gateway 2 replicas.

URL Shortener Microservices

Một case study nhỏ nhưng đủ lát cắt của môn SE361: DDD, database-per-service, REST, messaging, event-driven, outbox, cache, JWT, rate limit, circuit breaker, monitoring, load testing và Docker deployment.

Scalable

cache + async + stateless
edge

Resilient

rate limit + circuit breaker
+ outbox

Observable

logs + metrics + Grafana